



Problem A. A Stack of Gold

Source file name: Stackof.c, Stackof.cpp, Stackof.java, Stackof.py
Input: Standard
Output: Standard

Lt. Columbo, arguably the best detective in the world, was faced with a problem. He was asked: if he was placed in a room with some number of stacks of gold-colored coins, where all but one stack consisted solely of tungsten coins, and the remaining stack consisted solely of pure gold coins, how could he determine which stack had the pure gold coins? Oh, excuse me, there's just one more thing ... he was also told he is given a modern penny scale and one penny. A modern version of the penny scale operates as follows: when you deposit a penny and put an object (or objects) on the scale, the machine displays the weight on a digital display. The scale is extremely accurate, down to milligrams. Every coin is the same size and color, but the tungsten coins weigh 29260 mg each and the gold coins weigh 29370 mg each.

Being the genius that he is, Columbo came up with the solution to figure out which stack contained the gold coins using a single weighing (since he has only one penny for the penny scale). As an example, suppose there are four stacks of coins labeled 1 through 4. If he took one coin from stack 1, two coins from stack 2, three coins from stack 3 and four coins from stack 4, and weighed those 10 coins together on the penny scale, he could determine which stack had the gold coins. "How?" you might ask. Suppose all four stacks were tungsten. The total weight of the 10 coins would be 292600 mg. If, say, stack 1 was gold (one gold coin in the pile that was weighed), the total weight would be 292710 mg. If stack 2 was gold (two gold coins in the pile that was weighed), the total weight would be 292820 mg. So, if you know the number of coins being weighed and the total weight of those coins, you can determine which stack has the gold coins. Just as a reminder, given a number of stacks, s , the number of coins, c , being weighed will be:

$$c = \frac{s(s+1)}{2}$$

Input

Input consists of a single line containing two integers w s , where w ($87890 \leq w \leq 147774000$) is the weight in milligrams reported by the scale and s ($2 \leq s \leq 100$) is the number of coin stacks. The stacks are labeled 1 to s with i coins from stack i moved to the scale for $1 \leq i \leq s$. For given values of w and s , it will always be possible to determine which stack is composed solely of gold coins.

Output

Output consists of a single positive integer indicating which stack contains the gold coins.

Example

Input	Output
292930 4	3
147774000 100	100



Problem B. Average Substring Value

Source file name: Average.c, Average.cpp, Average.java, Average.py
Input: Standard
Output: Standard

Let s be a nonempty string consisting entirely of base-10 digits (0-9). If the length of s is n , number the digits $1, 2, 3, \dots, n$ from left to right, and for $1 \leq i \leq j \leq n$, let $s[i, j]$ denote the substring consisting of the digits from position i to position j , inclusive. (It follows that we are only considering *nonempty* substrings.) Assign a value to each substring that is simply equal to the largest digit in the substring. What is the average value of the substrings of s ?

Note that two different substrings may be identical (as strings), but for the purposes of this problem they are treated as distinct. For example, if $s = 1010$, then $s[1, 2] = s[3, 4] = 10$ are distinct substrings (both with value 1).

Input

The input is a single nonempty string, s , of base-10 digits. The length of s is at most 200000.

Output

Output a line containing the average value of the substrings of s . If the average is an integer, print the integer. If the average is a proper fraction, i.e., is equal to a/b , where a and b are positive integers and $a < b$, print this fraction in lowest terms, with a '/' symbol separating the numerator and denominator. If the average is greater than 1 and does not simplify to an integer, print the whole part followed by the proper fractional part, separated by a space, with the proper fractional part in lowest terms and formatted as described in the previous sentence.

Example

Input	Output
123	2 1/3
4084	6
1010	4/5
00000	0

Problem C. Balancing Art

Source file name: Balancing.c, Balancing.cpp, Balancing.java, Balancing.py
 Input: Standard
 Output: Standard

Pete Tencious is a world-renowned artist who specializes in mobiles. The San Francisco Museum of Art is currently displaying a collection of his artwork entitled Balance I, Balance II, Balance III, ..., you get the idea. Each of these works contains two or more spheres with wires strung between them. At the ends of each wire are a number of disks that are attached to the two spheres that the wire connects. When you sum up the number of disks at each sphere you get the same number (hence the word “Balance”) – Pete calls this the *balance number* for each mobile. On some of these wires there are also extra disks suspended between spheres. The artist says this work represents the balance of nature (the disks attached to the spheres) corrupted by the influence of humankind (the extra disks between the spheres). Clearly Pete has a much better impression of nature than people.

The funny thing is that Mother Nature decided to step in and throw Pete a curve. A minor earthquake in the Bay area has dislodged the disks, and they are now all hanging loose between spheres. Pete is currently working on Balance CCXLI and can’t be reached, so the museum curators have to fix the mobiles themselves. They can’t remember exactly what the balance number should be for each mobile, but they decide it should be as large as possible, leaving as few extra disks in between the spheres as possible (they apparently have a little better view of humankind than you-know-who). However, determining the minimum number of disks left between the spheres is a bit difficult, so they have come to you for help.

Figure 1 shows the state of one mobile after the earthquake, corresponding to the first example input. Figure 2 shows one way the disks could be moved by the curators so that each sphere has the same number of disks, while leaving the minimum number of disks between the spheres.

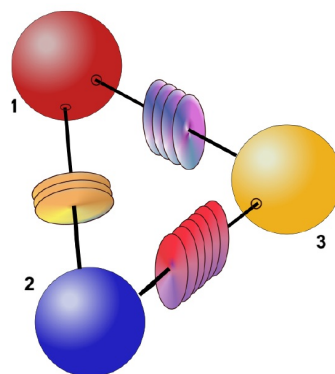


Figure 1. After the earthquake

Input

The first line of input contains two positive integers n m , where n ($2 \leq n \leq 200$) is the number of spheres in a mobile (numbered 1 to n) and m ($1 \leq m \leq 500$) is the number of wires connecting the spheres. This is followed by m lines of the form s_1 s_2 d , where s_1 and s_2 ($1 \leq s_1, s_2 \leq n$, $s_1 \neq s_2$) are two spheres connected by a wire and d ($0 \leq d \leq 10000$) is the number of disks hanging on the wire after the earthquake. There is at most one wire between any two spheres.

Output

Output the minimum number of disks left hanging on the wires between spheres after the maximum balance number has been reached.

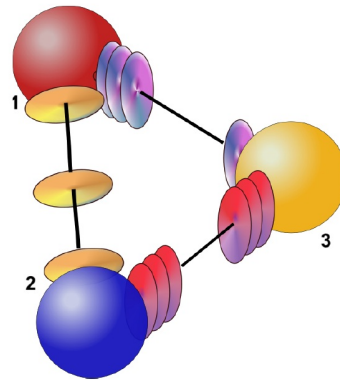


Figure 2. Disks moved to maximize balance number

Example

Input	Output
3 3 1 2 3 1 3 4 2 3 6	1
5 4 1 2 2 1 5 2 2 3 2 2 4 20	16

Problem D. Brownian Bears

Source file name: Brownian.c, Brownian.cpp, Brownian.java, Brownian.py
Input: Standard
Output: Standard

Dr. Ursula Major is an internationally renowned expert in the study of bears, specifically brown bears, which are known in many parts of North America as grizzly bears. She is most famous for her discovery of an extremely rare brown bear subspecies – the Brownian bear, whose feeding behavior seems to be guided by an intriguing mixture of regularity and randomness.



Image by Alex Taylor; Used with permission

At present, Dr. Major is studying a pair of Brownian bears living on a narrow strip of land that is oriented east-west. This territory has n evenly spaced locations where the bears can forage for berries, and Dr. Major has labelled these locations $1, 2, \dots, n$ from west to east. Every morning, each Brownian bear wakes up in one of the n locations and randomly chooses to move either east or west to the neighboring location. What is remarkable is that the probability of moving in either direction is exactly 50%! (Dr. Major's working theory is that this is rooted in some kind of quantum mechanical phenomenon in the bears' brains.) If a bear happens to start the day in one of the end locations (1 or n), then it will randomly choose between moving to the sole neighboring location or staying where it is. After making its choice, the bear spends the day foraging for berries in the chosen location, and then also sleeps there that night. The next morning, the process begins all over again.

Dr. Major is particularly interested in days when the two bears forage together, i.e., in the same location, so she plans to observe them over a period of time, possibly as long as a month (or until her funding runs out). At the beginning of the first observation day, the two bears wake up in different locations. Can you help Dr. Major determine the probability that the two bears will forage together for at least one day during the course of her experiment?

Note that the two bears can move past each other without foraging together. For example, if the bears start a particular day in locations 4 and 5 , and if the bear in location 4 moves east and the bear in location 5 moves west, then the bears simply pass each other that morning without foraging together.

Input

The input consists of a single line containing four integers, $n \ x \ y \ d$, where n ($2 \leq n \leq 100$) is the number of locations, x and y ($1 \leq x, y \leq n$, $x \neq y$) are the locations where the two bears wake up at the beginning of the first observation day, and d ($1 \leq d \leq 31$) is the number of observation days.

Output

Output a line containing the probability that the two bears forage together for at least one day during the d observation days. Express this probability as a fraction a/b in lowest terms, where a is a non-negative integer and b is a positive integer. See the example outputs.

Example

Input	Output
4 1 2 2	3/8
6 2 5 2	0/1
2 1 2 1	1/2

Problem E. Snakey String

Source file name: Snakey.c, Snakey.cpp, Snakey.java, Snakey.py
Input: Standard
Output: Standard

A *snakey string* is a fancy rendering of an otherwise normal string of text. When a string is made snakey, it is placed on a 2D grid such that the following conditions are met:

- The first character of the string is on the first column of the grid, the second on the second column, ..., and the last on the last column.
- Each character in the string occupies either the row directly above or directly below the row of the previous character.

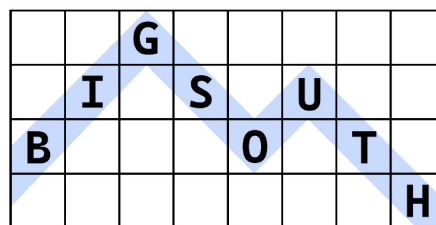


Figure 3. The snakey string in the example case.

Given a snakey string, can you recover the original string?

Input

The first line of input contains two integers r and c ($2 \leq r, c \leq 100$), the number of rows and columns of the grid.

The next r lines each contain c characters that form the 2D grid containing the snakey string. Empty cells are encoded with a period, while uppercase letters (A - Z) represent characters in the original string. Every column in the grid contains exactly one uppercase letter. It is possible that some rows do not contain any uppercase letters.

Output

Output a single string, the original string that was used to build this snakey string.

Example

Input	Output
4 8 ..G..... .I.S.U.. B...O.T.H	BIGSOUTH

Problem F. Leapfrog Encryption

Source file name: Leapfrog.c, Leapfrog.cpp, Leapfrog.java, Leapfrog.py
Input: Standard
Output: Standard

We've come up with a new encryption method we call Leapfrog Encryption. It is a key-based encryption scheme where an alphabetic key specifies how letters in the *plaintext* (the text to be encrypted) are placed in the *ciphertext* (the resulting encrypted string). Here's how Leapfrog Encryption works:

1. Remove all non-alphabetic characters from the plaintext and convert all remaining letters to lower-case.
2. Convert the letters of the key to their location in the alphabet +1 (so that 'a' converts to 2, 'b' converts to 3 and so on). This gives us a sequence of numbers $d_1, d_2, \dots, d_n$ where n is the length of the key.
3. Going left-to-right, place the first letters in the plaintext in every d_1 -th location of the ciphertext (where the length of the ciphertext is equal to the number of letters in the plaintext). So for example, if $d_1 = 5$ the first letter in the plaintext goes in position 5 of the ciphertext (numbering the first location in the ciphertext as position 1), the second letter in the plaintext goes in position 10 in the ciphertext, and so on.
4. Repeat this with d_2 but this time going right-to-left through the ciphertext, only counting the empty positions (leapfrogging over the letters already in the ciphertext).
5. Continue with d_3, d_4 , etc., alternating the direction you go through the ciphertext each time.
6. If there are still letters left in the plaintext after using d_n , fill in the remaining empty locations in the ciphertext with these remaining letters, again going in the opposite direction of the previous pass (this is equivalent to having $d_{n+1} = 1$).

For example, if our plaintext is "Send more monkeys!" and our key is "bea", the encryption proceeds as follows:

```

b → 3, left-to-right:  _ _ s _ _ e _ _ n _ _ d _ _ m
e → 6, right-to-left:  _ _ s _ _ e o _ _ n _ _ d _ _ m
a → 2, left-to-right:  _ r s _ _ e e o _ _ n m _ _ d o _ _ m
last pass, right-to-left: s r s y e e o e n m k d o n m

```

Decryption is done by ... hey, you know what? We're going to let you figure that out.

Input

The first line of input contains two strings t k where t is either E or D indicating whether to perform encryption or decryption and k is the lowercase alphabetic key. The length of k will be between 1 and 100, inclusive. The second and final line of input contains the plaintext to encrypt (if t is E) or the ciphertext to decrypt (if t is D). This string is non-empty and has a maximum length of 2000. A ciphertext string consists of lowercase letters only, while a plaintext string may contain uppercase letters, numbers, punctuation and spaces as well (all counted as part of the length of the string) and is guaranteed to contain at least one letter.

Output

Output the encrypted or decrypted text. Your output should only contain lowercase letters.



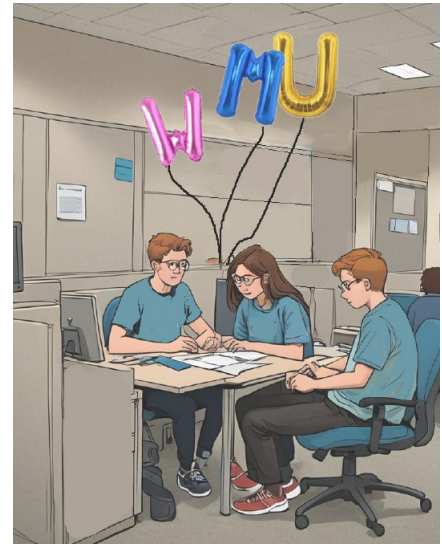
Example

Input	Output
E bea Send more monkeys!	srsyeeoenmkdonm
D bea srsyeeoenmkdonm	sendmoremonkeys
D zyxwvutsrqponmlkjihgfedcba lafogrpe	leapfrog

Problem G. Letter Balloons

Source file name: Balloons.c, Balloons.cpp, Balloons.java, Balloons.py
Input: Standard
Output: Standard

You are organizing a programming contest, and have decided that the first team to solve each problem will get a balloon in the shape of the problem's letter. For example, suppose there are twenty-three problems in the contest, labeled A through W. The team members for Wossa Motta University are hoping to be first to solve problems M, U, and W so that they can fly their university's initials above their programming station. In fact, a lot of teams have the same idea: try to be first-solvers of problems that spell out their school's abbreviation. It might be possible for both Wossa Motta U. and the Spittinyer Institution to achieve this goal, but neither will be able to do so if Muddinyer Institute manages to solve problems M and I before they do (we are assuming there will never be a tie for the first solution to any problem). On the other hand, Muddinyer I. can't achieve it if either Wossa Motta U. or Spittinyer I. solves M or I first. Schools like Toe Tac Tech are out of luck no matter what, since a team can get at most a single letter balloon for any problem. And Xerxes College is also out of luck because there is no problem X in this example.



NightCafe

You've been wondering—what is the maximum number of teams that can proudly display their school's initials with “first-solver” balloons at the end of the contest?

Input

The first line of input contains two integers p t , where p ($1 \leq p \leq 26$) is the number of problems in the contest, and t ($1 \leq t \leq 20$) is the number of teams. Problems are labeled with the first p letters of the alphabet. Each of the following t lines contains a nonempty string of at most 80 uppercase letters describing a school's initials. There is only one team from each school, but several schools may have the same initials.

Output

Output a single integer consisting of the maximum number of teams that can be first solvers of all the problems that form the initials of their school name.

**Example**

Input	Output
23 5 WMU SI MI TTT XT	2
6 6 ABC BDE ABE BF BF CEF	1

Problem H. Moonlit Time Machine

Source file name: Moonlit.c, Moonlit.cpp, Moonlit.java, Moonlit.py
 Input: Standard
 Output: Standard

After decades of research in alchemy and magic, you have finally created your magnum opus – a room that, when bathed in the glow of the moon, sends you several days into the future. You have given this room a very classy name: the *Moonlit Time Machine*.

Whenever you use the time machine, you are projected forward some number of days. You don't know the number, but you know that it is between 1 and 28 (inclusive), and that the number is always the same every time you use the machine. As an alchemist, you wish to find out this number! While there is no technology to tell you the date, you can look at the moon's cycle. You know that every 28 days, the moon cycles through a list of phases, as defined below:



Day(s)	Phase
0	New
1 to 4	Crescent
5 to 8	Quarter
9 to 13	Gibbous
14	Full
15 to 19	Gibbous
20 to 22	Quarter
23 to 27	Crescent

To figure out this number of days, you wait until you see a new moon, then use the time machine several times, recording the moon's phase after each use. Given your observations, can you determine a possible number of days that the Moonlit Time Machine sends you forward each time?

Input

The first line of input contains a single integer n ($3 \leq n \leq 100$), the number of observations you have made.

Each of the next n lines contains a single string describing an observed phase of the moon. Each string is one of **New**, **Crescent**, **Quarter**, **Gibbous**, or **Full**, with its first letter capitalized and the remaining letters lowercase. It is guaranteed that these are valid observed phases of the moon after using your time machine.

Output

Output a single integer, the number of days the Moonlit Time Machine sends you into the future with each use. If there are multiple answers, output the smallest one. It is guaranteed that at least one valid answer exists.



Example

Input	Output
4 Crescent Quarter Gibbous Gibbous	3



Problem I. Oooh I See

Source file name: Ooohisee.c, Ooohisee.cpp, Ooohisee.java, Ooohisee.py
Input: Standard
Output: Standard

Captain O'Capten has hidden some treasure and created a map to mark where it is buried. Rather than using 'X' to mark the spot, he has decided to obfuscate the location by using a grid of uppercase O (the letter O) characters and 0 (the number 0) characters. The treasure's position is given by the location of a 0 character surrounded on all sides by 8 O characters. That is, a 0 character with an O immediately above, below, to the left, to the right, diagonally above to the left, diagonally above to the right, diagonally below to the left, and diagonally below to the right.

Captain O'Capten wants to recover the location of his treasure but he is finding his map hard to read (huh, go figure). Help Captain O'Capten find the location of his treasure, or exclaim **Oh no!** if such a location is not marked on the map or there is more than one such location.

Input

The first line contains two integers r and c ($1 \leq r, c \leq 50$), where r indicates the number of rows and c indicates the number of columns of characters in the map. Rows are numbered 1 to r , top to bottom, and columns are numbered 1 to c , left to right. This is followed by r lines, each with c characters, where each character is either O or 0.

Output

If there is no 0 character surrounded on all sides by 8 O characters, then output one line consisting of the exclamation, **Oh no!**. If there is more than one 0 character surrounded on all sides by 8 O characters, then output one line consisting of the exclamation, **Oh no! N locations**, with the number of locations instead of N . If there is exactly one 0 character surrounded on all sides by 8 O characters, then output one line containing two integers. The first integer is the index of the row of the 0 character and the second integer is the index of the column of the 0 character.

Example

Input	Output
3 6 000000 000000 000000	2 5
5 3 000 000 000 000 000	Oh no! 2 locations
4 4 0000 0000 0000 0000	Oh no!

Problem J. Pascal Meets Boole

Source file name: Pascal.c, Pascal.cpp, Pascal.java, Pascal.py
Input: Standard
Output: Standard

Many people are familiar with Pascal's Triangle, a triangular arrangement of integers named after the French mathematician and philosopher Blaise Pascal (1623-1662). If we number the rows of Pascal's Triangle 1, 2, 3, ..., starting from the top, then row r contains r elements, which we will number 1, 2, ..., r from left to right. The 1st and r th elements in row r are set equal to 1, and for $r \geq 3$ and $1 < i < r$, element i in row r is the sum of elements $i - 1$ and i in row $r - 1$. More informally, each "non-edge" element is the sum of the two elements immediately above it. Figure 4(a) depicts the first 8 rows of Pascal's Triangle.

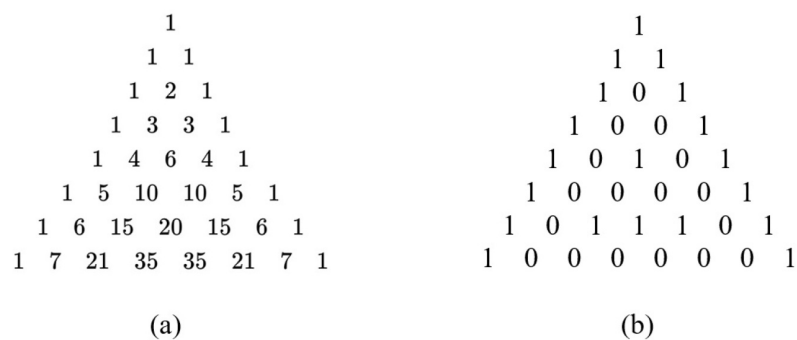


Figure 4. (a) Pascal's Triangle, (b) Pascal-Boole triangle for function 1000

But what if we consider a rule other than standard addition for combining values? Since the edge elements are bits (1's), a natural option is to use any two-input Boolean function, named after the English mathematician and philosopher George Boole (1815-1864). For example, the Boolean function given by the following truth table generates the triangle depicted in Figure 4(b) (where we also show the first 8 rows). In this truth table, x and y correspond to elements $i - 1$ and i , respectively, in row $r - 1$, and $f(x, y)$ is the resulting element i in row r .

x	y	$f(x, y)$
0	0	1
0	1	0
1	0	0
1	1	0

In general, if we label the bits in the rightmost column of any such truth table b_{00} , b_{01} , b_{10} , b_{11} from top to bottom, then we can compactly represent a two-input Boolean function by the 4-bit string $b_{00}b_{01}b_{10}b_{11}$. So the example function above is represented by 1000.

Your challenge is to answer two kinds of questions about "Pascal-Boole" triangles:

1. For a given Boolean function, f , what is the bit in row r , position i ?
2. For a given Boolean function, f , how many 1's are there in the first r rows?

Input

The first line of input contains an integer, n ($1 \leq n \leq 250$), the number of test cases. This is followed by n lines, each of which has one of two forms:



1. $f \text{ B } r \ i$
2. $f \text{ N } r$

In both cases, f is a 4-bit binary string representing a two-input Boolean function, and r is an integer ($1 \leq r \leq 10^6$). In the first case, i is an integer ($1 \leq i \leq r$).

Output

For a test case of the form $f \text{ B } r \ i$, output a line containing the bit in row r , position i of the Pascal-Boole triangle generated using f . For a test case of the form $f \text{ N } r$, output a line containing the number of 1's in the first r rows of the Pascal-Boole triangle generated using f .

Example

Input	Output
3	1
1000 B 5 3	28
1111 N 7	0
0100 B 6 4	

Problem K. Pony-less Express

Source file name: Ponyless.c, Ponyless.cpp, Ponyless.java, Ponyless.py
 Input: Standard
 Output: Standard

Howdy, pardner! You've been put in charge of mail delivery in these here parts. Your headquarters is in Capital City, and Miss Penelope has asked us to tell all of the local farmsteads about her fashionable doin's. The roads hereabouts were built so that each farmstead connects to Capital City by only one set of roads, which may connect through multiple other farmsteads. Each road takes exactly one day to travel on a horse.

Each farmstead wants to receive news on their own specific timetable (so as not to distract the hired hands from their work), and gets peeved if the delivery misses the desired date, either early or late. Specifically, each farmstead i has a perfect date D_i for receiving news. If the news arrive on day d , the anger level at the farmstead is $C_i(D_i - d)^2$. Obviously, we want to minimize the total anger level over all the farmsteads, so we need to get on our horses and send this important information to each farmstead as close to their desired date as possible! The only problem is ... well ... we actually don't HAVE any horses. Instead, we can requisition exactly one horse from each location (farmstead or Capital City) each day, ride it for 1 day, and let it return to its starting location at the end of the day (don't worry, these horses know how to find their way back). The next day, another rider can take that horse and travel to a different location (if necessary). This process repeats in all of the farmsteads as well as Capital City.

Oh, and one more thing: we ain't payin' riders to sit idle at any farmstead, gittin' into who knows what kind of trouble. So if news arrives at farmstead i on day d and there are m roads leading to m other farmsteads that have not received any news yet, a horse will leave farmstead i on each day $d + 1, d + 2, \dots, d + m$ until every neighboring city has been visited, even if waiting extra days might lead to lower anger levels for some of the farmsteads.

For example, consider the layout of farmsteads shown in Figure 5 (corresponding to Example Input 1), where the C_i, D_i values are shown to the left of each farmstead. The optimal way to deliver the news is as follows:

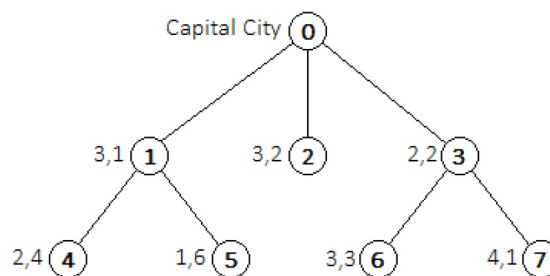


Figure 5. Example layout.

- Day 1: send a horse from Capital City to farmstead 3
- Day 2: send a horse from Capital City to farmstead 1 and a horse from farmstead 3 to farmstead 7
- Day 3: send a horse from Capital City to farmstead 2, a horse from farmstead 1 to farmstead 4 and a horse from farmstead 3 to farmstead 6
- Day 4: send a horse from farmstead 1 to farmstead 5



The total anger level, adding up from farmstead 1 to 7, is $3(1-2)^2 + 3(2-3)^2 + 2(2-1)^2 + 2(4-3)^2 + 1(6-4)^2 + 3(3-3)^2 + 4(1-2)^2 = 3 + 3 + 2 + 2 + 4 + 0 + 4 = 18$.

Can you help us figure out just how angry people will be in total, so we know what we're in for?

Input

The first line of input contains a positive integer n ($n \leq 200$) indicating the number of farmsteads, numbered 1 to n , with Capital City labeled 0. Following this are n lines each containing three integers $c d r$ ($1 \leq c \leq 20$, $1 \leq d \leq n$, $0 \leq r \leq n$), where the i^{th} of these lines specify the C_i and D_i values for the i th farmstead and r indicates that there is a road from farmstead i to farmstead r (or Capital City if $r = 0$). The roads are set up so that there is a unique path between any two farmsteads, and between each farmstead and Capital City.

Output

Output the minimum anger level that can be achieved assuming the first horse leaves Capital City and arrives at a first farmstead on day 1.

Example

Input	Output
7 3 1 0 3 2 0 2 2 0 2 4 1 1 6 1 3 3 3 4 1 3	18
3 5 2 2 4 1 0 6 3 1	0

Problem L. Repetitive Routes

Source file name: Routes.c, Routes.cpp, Routes.java, Routes.py
Input: Standard
Output: Standard

Tory operates a dial-in mobility service where customers can book a vehicle to come and pick them up from one location and drive them to another location to be dropped off. The vehicles used can seat many passengers, so sometimes additional stops are made along the way to pick up and drop off other passengers. Users of the service are generally tolerant of some inefficient routes, but they are less tolerant of revisiting a location that they have visited at least once on their journey already. Tory has come up with a sequence of pickups and drop offs to serve all of the passengers but he wants to know how many complaints he is likely to receive. From past experience, Tory knows that he can expect one complaint for each time a customer returns to a location that they have already been to on their journey. Note that this means that the same customer may lodge multiple complaints, and possibly for visiting the same location more than two times! Given a sequence of pickups and drop offs and their locations, determine how many complaints Tory can expect to receive.

The customer's pickup location and drop off location both count towards locations that they may return to and complain about. Consecutive pickups or drop offs at the same location also count as repeated visits to the same location for anyone in the vehicle at the time. The customer's drop off location may be the same as their pickup location. As per above, the customer will complain about this!

Input

The first line of input is the number of customers, n ($1 \leq n \leq 200000$). This is followed by $2n$ lines, each consisting of two integers. The first is a customer number from 1 to n and the second represents a location and is in the range 1 to $2n$. Distinct location numbers correspond to distinct locations. Each customer number will appear exactly twice. The first occurrence of customer number C corresponds to the pickup of customer C and the second occurrence of customer number C corresponds to the drop off. All lines in between correspond to locations visited and pickups and drop offs completed while customer C is in the vehicle. Customer 1 will be the first customer to be picked up. Customer C will be picked up before customer $C + 1$ is picked up. Similarly, location 1 will be the pickup location of customer 1 and a location $L + 1$ will only be visited if location L has been visited. There is no limit on the number of passengers that can be in the vehicle at the same time.

Output

Output a single integer, the number of complaints Tory can expect to receive for the sequence of pickups and drop offs.



Example

Input	Output
4 1 1 2 2 2 3 3 2 4 1 4 2 1 3 3 4	5
4 1 1 2 1 3 1 4 1 4 1 3 1 2 1 1 1	16

Problem M. Smart Password Validation

Source file name: Password.c, Password.cpp, Password.java, Password.py
Input: Standard
Output: Standard

People often make errors when entering passwords for one of their many accounts. This is becoming more of a problem with the rather lengthy passwords that many online portals require (the more characters in the password, the more chance there is of making a typing mistake). Practitioners of the “touch typing” technique are prone to make serious errors if one of their hands is not in the proper home row position when typing, since touch typists do not need to look at the keyboard as they type. This can cause all characters typed by a misplaced hand to be “one key off”. E.g., on a QWERTY keyboard, “lion” might become “kuib” if the right hand is placed one key to the left of normal home position.

The Frobozz Magic Security Company has decided to implement a new password validation algorithm that takes into account some common mistakes (but not all) made by folks when entering passwords on a QWERTY-style keyboard (see Figure 6).

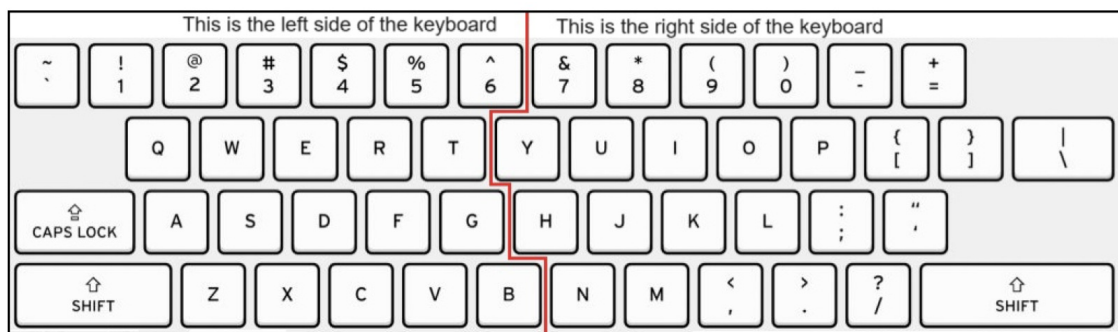


Figure 6. QWERTY keyboard

The effect of pressing CAPS LOCK is limited to alphabetic characters and toggles the effect of the shift key for those characters. In others words, when CAPS LOCK is not in effect, all letters are typed as lowercase unless one or both shift keys are pressed to produce an uppercase letter. The opposite behavior occurs when CAPS LOCK is on—uppercase is the default unless one or both shift keys are pressed, producing lowercase letters. The algorithm designers assume that the state of the CAPS LOCK key will not change in the middle of entering a password.

Their algorithm looks for deviations from a correct sequence of keystrokes. For example, if the correct password is “ALg*”, the correct sequence, using the keyboard above, is “SHIFT-A, SHIFT-L, G, SHIFT-8”. If the user were to accidentally hit the CAPS LOCK key before entering the password, but then used the correct keystroke sequence, they would end up typing “aLg*”. Likewise, if a touch typist accidentally placed the fingers of the left hand one space to the right of the correct position, made no other errors, and attempted to follow the correct sequence (unaware of the shift in finger positions), they would type “SLh*”. And if, in addition, they also accidentally activated the CAPS LOCK key before entering the password, they would type “sLH*”. Finally, a user might accidentally insert or delete a character. For example, the password “ALg*” might be mistyped as “AL*” or “ALLg*”.

The Frobozz Magic Security Company has decided to permit certain deviations, or combinations of deviations, from the correct sequence, as described in Figure 7. At most one of the LS, RS, EC and MC errors is permitted, but any of these may occur in conjunction with a CL error.

Name	Acronym	Description	Examples
Left side off by one	LS	All characters on the left side of the keyboard are shifted right by one key. The left-shift key shifted right by one key is still the left-shift key.	FlatHead $\Rightarrow$ GlSyHrsf
Right side off by one	RS	All characters on the right side of the keyboard are shifted left by one key. The right-shift key shifted left by one key is still the right-shift key.	FlatHead $\Rightarrow$ FkatGead
Single extra character	EC	The entered password has an extra character	Zorkmid $\Rightarrow$ Zorkmiid (extra i)
Single missing character	MC	The entered password is missing a character.	FCD#3 $\Rightarrow$ FCD3 (missing the #)
CAPS LOCK was in effect by mistake	CL	All lowercase letters appear as uppercase and all uppercase letters appear as lowercase	WE do it! $\Rightarrow$ we DO IT!

Figure 7. Permitted Error Types

Input

There are multiple lines of input. The first line contains a string, p , which is the correct password ($2 \leq \text{length}(p) \leq 24$). The second line contains a single positive integer n ($1 \leq n \leq 1000$), which is the number of lines that follow. Each of the following n lines contains a single password to test, t_i ($\text{length}(p) - 1 \leq \text{length}(t_i) \leq \text{length}(p) + 1$). The characters for all passwords come only from those that appear on the QWERTY keyboard picture above, as well as the lowercase letters.

Output

Output consists of n lines. For each t_i the line should consist of the word **YES** if t_i matches the password p using Frobozz's algorithm, **NO** if it does not.

Example

Input	Output
Password='A:'	YES
9	YES
Password='A:'	YES
pASSWORD='a:'	YES
Psddeotf=1S:	YES
oASSWIRD-'al;	YES
Password='A:'x	YES
pASWORD='a:'	NO
pASSWORD='a:'X	NO
Osddeitf-1SL;	
Psdeotf=1S:	